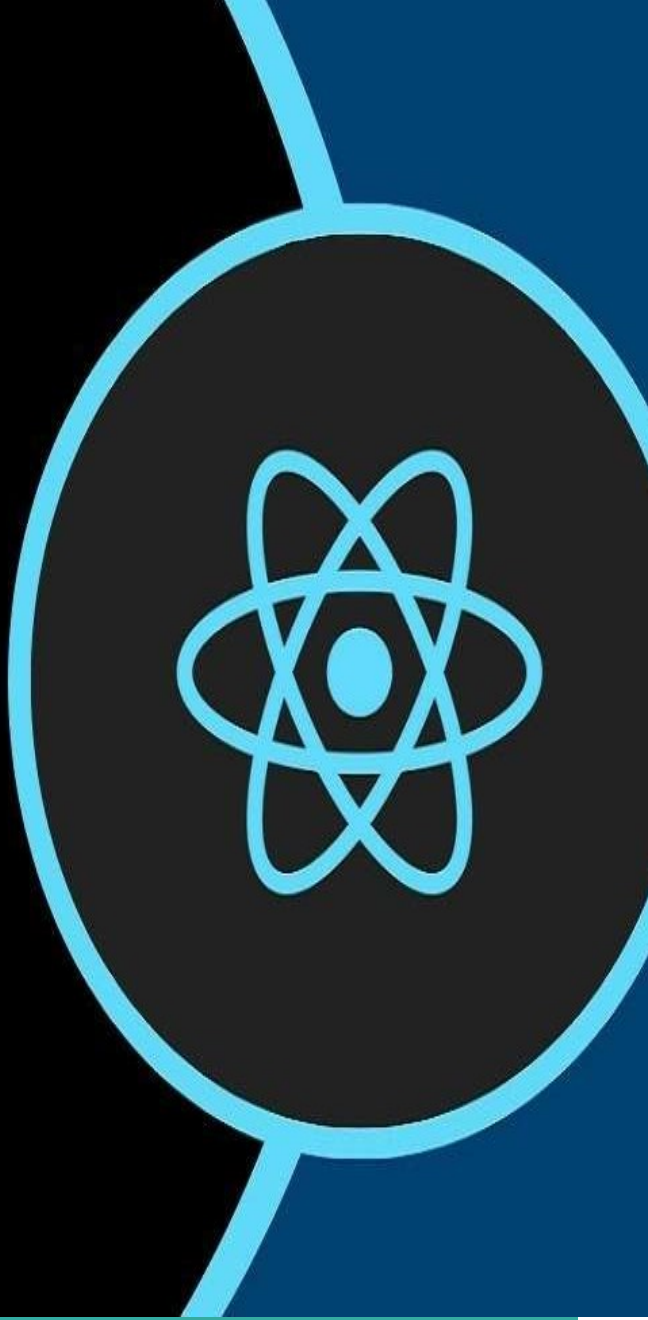


JavaScript-13

Introduktion till React

30 september 2026

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

Introduktion till React JS

Hittills i kursen har vi byggt webbsidor genom att manuellt leta upp element (querySelector), skapa nya taggar (createElement), ändra text (textContent) och trycka ut dem på sidan (appendChild). Detta kallas för Imperativ programmering – vi dikterar exakt hur webbläsaren ska gå tillväga i varje litet steg.

När en webbapplikation växer och får hundratals rörliga delar (tänk dig Facebook-feeden med likes, kommentarer, chattfönster och aviseringar som uppdateras samtidigt) blir detta pussel snabbt ohanterligt. Det är lätt att missa att uppdatera ett element, vilket leder till dolda buggar.

För att lösa detta skapade Facebook React JS. React introducerar ett helt nytt tankesätt: Deklarativ programmering och Komponentbaserad arkitektur.

Imperativ vs. Deklarativ kod

Skillnaden mellan hur vi har kodat tidigare och hur vi kodar i React kan sammanfattas så här:

- **Imperativt (Vanlig JS):** Du ger datorn en detaljerad instruktionsmanual.
"Leta upp knappen, kontrollera om användaren är inloggad, om ja: ändra klassen till grön, skapa en p-tag, lägg till texten..."
- **Deklarativt (React):** Du beskriver bara slutresultatet baserat på din data.
"Om isLoggedIn är true, visa en grön knapp. Annars visa en grå knapp."

Du ändrar bara på din data, och React sköter automatiskt hela pusslet med att uppdatera skärmen åt dig.

Virtual DOM – Hur React är så snabbt

Att ändra i den riktiga HTML-DOM:en är det absolut långsammaste en webbläsare kan göra. Om du ändrar ett litet element i en stor lista måste webbläsaren ofta räkna om layouten för hela sidan.

React löser detta genom att skapa en **Virtual DOM** (en virtuell kopia av HTML-trädet som ligger lagrat i datorns snabba RAM-minne).

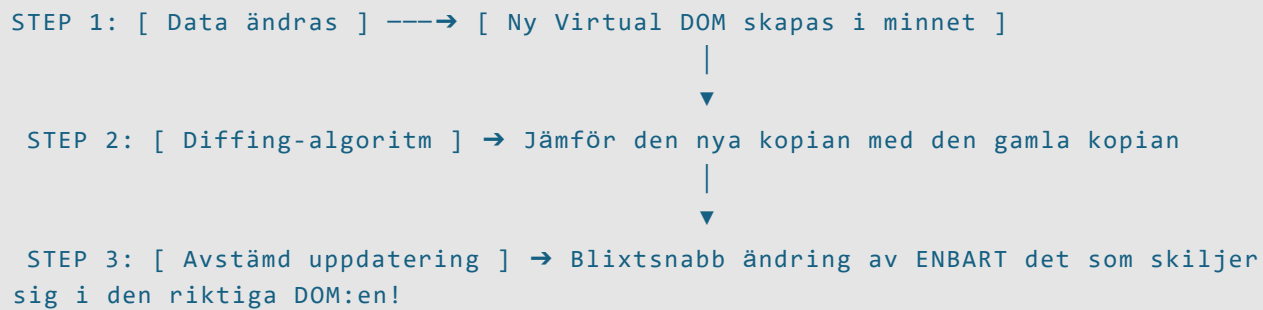


Bild 1: Istället för att rita om hela sidan, räknar React ut exakt vilken minsta lilla punkt som har ändrats och uppdaterar enbart den i webbläsaren.

Komponenter – Webbsidans Lego-bitar

I React delar vi upp webbsidan i små, självständiga och återanvändbara bitar som kallas Komponenter. En komponent är i grunden en helt vanlig JavaScript-funktion som returnerar hur en del av skärmen ska se ut. Man skapar en komponent för ett sökfält, en för menyrad och en för produktkort, och sedan pusslar ihop dem.

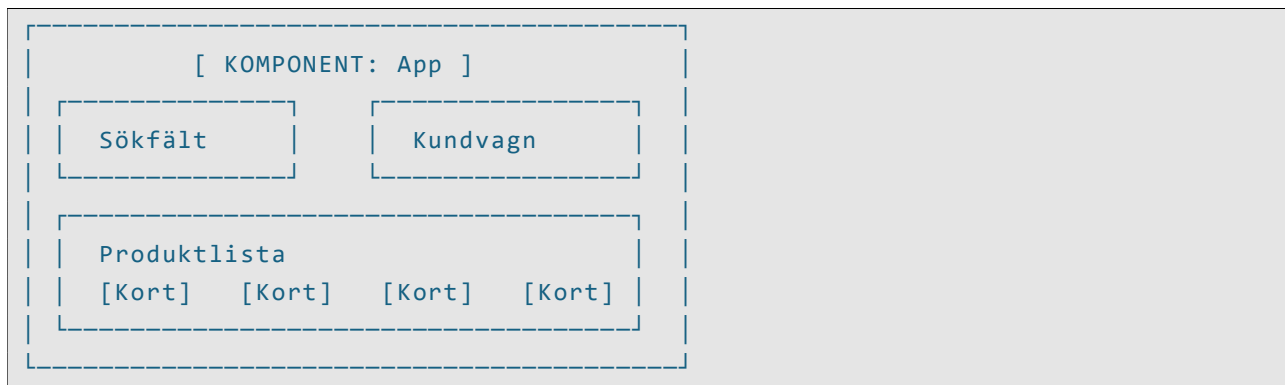


Bild 1 En React-applikation är uppbyggd av ett träd med komponenter som sitter inuti varandra.

JSX – HTML direkt i JavaScript

Det första man reagerar på när man ser React-kod är att det ser ut som en blandning av JavaScript och HTML. Detta kallas för JSX (JavaScript XML).

JSX gör att vi kan skriva HTML-strukturen direkt inuti våra JavaScript-funktioner. Det är otroligt smidigt, eftersom vi slipper hålla på med innerHTML eller strängar.

Här är en enkel React-komponent. Lägg märke till att det bara är en helt vanlig JavaScript-funktion som returnerar något som ser ut som HTML

```
// En enkel React-komponent (måste starta med STOR bokstav!)
function VälkomstHälsning() {
  const användarNamn = "Anna";

  return (
    <div className="halsning-box">
      <h1>Välkommen tillbaka, {användarNamn}!</h1>
      <p>Idag är en bra dag att koda React.</p>
    </div>
  );
}
```

Bild 2 Med måsvingar {} kan vi hoppa direkt från HTML-läge till JavaScript-läge.

State (useState) – Komponentens minne

Komponenter är inte bara statiska Lego-bitar, de måste kunna ändra sig. Om en användare klickar på en knapp vill vi kanske att en siffra ska räknas upp, eller att en meny ska öppnas.

För att en komponent ska kunna komma ihåg saker använder vi något som kallas State (tillstånd). Vi skapar ett state med hjälp av en inbyggd React-funktion som heter useState.

När värdet i ett state ändras, fattar React det direkt och ritar om just den komponenten på skärmen blixtnabbt.

Här är exakt samma funktionalitet som vi byggde med en tung Closure-funktion i Labb 08, fast skrivet på "React-sättet".

```
import { useState } from 'react';

function KlickRäknare() {
  // antal = variabeln som håller värdet (startar på 0)
  // setAntal = funktionen vi MÅSTE använda för att ändra värdet
  const [antal, setAntal] = useState(0);

  return (
    <div className="raknare-container">
      <h2>Du har klickat {antal} gånger</h2>

      {/* När man klickar anropar vi setAntal och plussar på 1 */}
      <button onClick={() => setAntal(antal + 1)}>
        Klicka här!
      </button>
    </div>
  );
}
```

Bild 3 Exempel på hur man kan lägga till ett onClick-event direkt på ett element.

Istället för att vi manuellt letar upp ett <h2>-element med `querySelector` och ändrar dess text, uppdaterar vi bara vårt State. React ser till att skärmen uppdateras automatiskt.

Loopa ut data med .map()

Kommer du ihåg .map() från kapitel 06? I React används .map() hela tiden! Det är det absoluta standard-sättet att omvandla en array med data (t.ex. objekt från ett API) till en lista med HTML-komponenter på skärmen.

```
function Inköpslista() {
  // Här är vår data (sakerna vi ska köpa)
  const varor = [
    { id: 101, namn: 'Mjölk' },
    { id: 102, namn: 'Bröd' },
    { id: 103, namn: 'Kaffe' }
  ];

  return (
    <ul>
      {/* Vi loopar igenom varorna med .map() */}
      {varors.map((vara) => (
        <li key={vara.id}>
          {vara.namn}
        </li>
      ))}
    </ul>
  );
}
```

Bild 4 Här kan man använda sig av en smart-Array och loopa igenom den.

⚠ VIKTIGT !

React kräver att varje element i en loop har en unik key-egenskap (ett ID) så att React kan hålla reda på exakt vilket element som ändras, flyttas eller tas bort om datan uppdateras.

Så fungerar det i verkligheten

Tänk dig att du är en lärare och har en klass med 20 elever. Du ska dela ut veckobrev.

Om alla elever har likadana tröjor och saknar namnlappar, blir det jättesvårt för dig att veta vem som är vem på långt håll. Om elev nummer 5 byter plats med elev

nummer 12, måste du gå fram och titta noga på varje person för att räkna ut vem som flyttade på sig. Det tar tid och energi.

Men om varje elev har en **unik** namnlapp på bröstet (en **key**), ser du direkt: *"Aha, det var Johan som flyttade dit, och Sara som gick ut ur rummet."*

Så fungerar det i React:

- **Loopen** är klassrummet med alla elever (t.ex. en lista med produkter i en varukorg).
- **Varje element** är en enskild elev (en produkt).
- **key-egenskapen** är namnlappen.

När något ändras i din lista – till exempel att du tar bort en produkt eller ändrar ordning på dem – tittar React på namnlapparna (key). Tack vare dem slipper React läsa igenom och bygga om hela listan från början. Den uppdaterar **bara just den saken** som ändrades.

Det gör att hemsidan laddar mycket snabbare och inte gör onödigt arbete!

Förklaring av kodexemplet

1. **key={vara.id}**: Detta är själva namnlappen. För Mjök blir key 101, för Bröd blir den 102, och så vidare.
 2. **Varför står det inte i webbläsaren?** Namnlappen är bara till för Reacts interna system. Den syns inte för personen som besöker hemsidan.
 3. **Vad händer om vi tar bort "Bröd"?** React kollar på listan, ser att namnlapp 102 är borta, och plockar blixtnabbt bort just det elementet från skärmen. Den rör inte Mjök (101) eller Kaffe (103).
-